1 Asymmetric-Key Cryptography and RSA ▸

▸ **Likely this year:** New this year, so expect one Forouzan-style numerical: "p, q, e given: find n, $\phi(n)$, d; encrypt P; decrypt C" (4–6 marks), plus a "why" question (proof of correctness, why $e = 2$ fails, one attack).

1.1 Why asymmetric-key cryptography exists ▸

▸ **Likely this year:** "Differentiate symmetric-key and asymmetric-key cryptography" / "Why is symmetric cryptography still needed?" (2–3 marks).

In symmetric-key cryptography both parties use the same secret key, so before Alice and Bob can talk privately they must already share a secret. This is the **key distribution problem**: the key has to travel over a secure channel that usually does not exist, and in a group of N users every pair needs its own key, so $N(N-1)/2$ secret keys must be created and stored (4950 for 100 users). Symmetric cryptography also cannot prove who sent a message, because anyone who can decrypt can also encrypt.

Asymmetric-key (public-key) cryptography removes the shared secret. Bob generates a **key pair**: a **public key** that he announces to everyone and a **private key** that only he knows. The public key locks; the private key unlocks. Anyone can encrypt for Bob with his public key, but only his private key decrypts, so Alice needs no earlier contact with Bob and N users need only N key pairs. Formally $C = f(K_{\text{public}}, P)$ and $P = g(K_{\text{private}}, C)$, where f is used only for encryption and g only for decryption, and knowing f must not reveal g. The secret is now carried by the receiver alone; for Bob to reply, he uses *Alice's* public key. Plaintext and ciphertext are treated as **numbers**, and encryption is a number-theoretic function rather than substitution and permutation of symbols.

Asymmetric cryptography does **not** replace symmetric cryptography, because it is far slower: one RSA operation is a modular exponentiation with 1024–2048-bit numbers, while AES needs a few table lookups and XORs per byte (RSA is roughly a thousand times slower). It is therefore used where symmetric cryptography cannot work: **exchanging a secret session key, authentication and digital signatures**. In practice the two are combined: a random AES key is sent encrypted under the receiver's RSA key, and the bulk data is encrypted with AES under that key.

	Symmetric-key	Asymmetric-key
Keys	one shared secret key	public/private key pair
Keys for N users	$N(N-1)/2$	N pairs
Secret held by	both parties	receiver only
Based on	substitution, permutation	number theory (factoring, discrete log)
Speed	fast (bulk data)	slow (keys, signatures)
Examples	DES, AES	RSA, ElGamal, ECC

1.2 Trapdoor one-way functions ▸

▸ **Likely this year:** "Define a trapdoor one-way function; give the RSA example (2 marks).

The design problem is to publish f while keeping f^{-1} hidden. A **one-way function** (OWF) is a function $y = f(x)$ that is easy to compute, while computing $x = f^{-1}(y)$ from y is computationally infeasible. A **trapdoor one-way function** (TOWF) has a third property: given y *and a secret* (the trapdoor), x is easy to compute. A public-key cipher needs exactly this: the public key lets everyone evaluate f, and the trapdoor is the private key.

Multiplication is one-way. For large primes p, q, computing $n = p \times q$ is easy, but recovering p and q from n is the **factorization problem**, for which no polynomial-time (classical) algorithm is known.

$y = x^k \bmod n$ **is a trapdoor one-way function.** Given x, k and n, fast exponentiation (§**??**) computes y in about $2 \log_2 k$ multiplications; given only y, k and n, taking the k-th root modulo n is believed to be as hard as factoring n. The trapdoor is k' with $kk' \equiv 1 \pmod{\phi(n)}$: then $x = y^{k'} \bmod n$. Computing k' needs $\phi(n) = (p-1)(q-1)$, and computing $\phi(n)$ needs p and q, so the trapdoor exists only for whoever chose the primes. RSA is this function with $k = e$ and $k' = d$.

1.3 RSA: key generation, encryption, decryption ▸

▸ **Likely this year:** Write the three algorithms, then run them on small primes.

RSA (Rivest, Shamir, Adleman, 1977) is the most widely used public-key cryptosystem. Bob builds the trapdoor from two primes and publishes only their product; Alice encrypts by raising the plaintext to the public exponent, and Bob decrypts by raising the ciphertext to the private exponent.

★ MUST KNOW : RSA

Key generation (Bob)

1. Choose two large primes $p \neq q$; compute $n = p \times q$ and $\phi(n) = (p-1)(q-1)$.
2. Choose e with $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.

3. Compute $d = e^{-1} \bmod \phi(n)$ with the extended Euclidean algorithm, so $e \times d \equiv 1 \pmod{\phi(n)}$.
4. Public key (e, n), announced. Private key d, kept secret (with p, q, $\phi(n)$).

Encryption (Alice): $C = P^e \bmod n$, with $0 \le P < n$. **Decryption (Bob):** $P = C^d \bmod n$.

Encryption and decryption are each one fast exponentiation, so both are polynomial in the size of n. Eve sees C, e and n and must compute $P = \sqrt[e]{C} \bmod n$, which is exponential for her unless she can factor n. The plaintext must be smaller than n, because the result is only known modulo n: two plaintexts that differ by n give the same ciphertext. A long message is cut into blocks, each a number less than n.

1.4 Why decryption works: proof of correctness ▸

▸ **Likely this year:** "Prove that RSA decryption recovers the plaintext" (2–3 marks).

Since d is the inverse of e modulo $\phi(n)$, $ed = 1 + k\,\phi(n)$ for some integer k. Decryption computes

$$C^d \equiv (P^e)^d = P^{ed} = P^{1+k\phi(n)} = P \cdot \left(P^{\phi(n)}\right)^k \pmod{n}.$$

Case 1: $\gcd(P, n) = 1$ (almost every P). Euler's theorem (§**??**) gives $P^{\phi(n)} \equiv 1 \pmod{n}$, so $C^d \equiv P \cdot 1^k = P \pmod{n}$. Since $0 \le P < n$, the reduced result is exactly P, not just something congruent to it.

Case 2: $\gcd(P, n) \neq 1$. Euler's theorem does not apply, but the result still holds. Since $0 < P < pq$, P is a multiple of exactly one prime, say $p \mid P$ and $q \nmid P$. Modulo p, $P \equiv 0$, so $P^{ed} \equiv 0 \equiv P$. Modulo q, Fermat's little theorem gives $P^{q-1} \equiv 1$, and $ed = 1 + k(p-1)(q-1)$, so $P^{ed} = P \cdot (P^{q-1})^{k(p-1)} \equiv P \pmod{q}$. Since $P^{ed} \equiv P$ modulo both p and q, and p, q are coprime, the CRT (§**??**) gives $P^{ed} \equiv P \pmod{pq}$. (This is Euler's "second version", $P^{k\phi(n)+1} \equiv P \pmod{n}$ for $n = pq$ and every P.) Check: $n = 221 = 13 \cdot 17$, $e = 5$, $d = 77$, $P = 65 = 5 \cdot 13$: $65^5 \bmod 221 = 182$ and $182^{77} \bmod 221 = 65$ ✓. Such a P is no practical concern, because finding one would give $\gcd(P, n) = p$ and factor n.

The two algebraic structures. RSA works in two places (§**??**). Encryption and decryption use the **public ring** $R = \langle \mathbb{Z}_n, +, \times \rangle$: P and C can be any element of $\mathbb{Z}_n$ (Case 2 shows non-invertible elements also work), and exponentiation is repeated multiplication, which anyone can do since n is public. Key generation uses the **private group** $G = \langle \mathbb{Z}^*_{\phi(n)}, \times \rangle$: e must belong to this group, i.e. $\gcd(e, \phi(n)) = 1$, so that its inverse d exists in it. Only Bob can build G, because knowing $\phi(n)$ is as good as knowing the factors: $p+q = n-\phi(n)+1$, and p, q are the roots of $x^2-(p+q)x+n = 0$. For $n = 187$, $\phi = 160$: $p + q = 28$, $x^2 - 28x + 187 = 0$ gives $x = (28 \pm 6)/2 = 17, 11$.

1.5 The slide examples ▸

▸ **Likely this year:** Verify one row by hand, e.g. "with $n = 77$, $e = 13$, find d and encrypt 5.

The lecture uses the key $p = 7$, $q = 11$, $e = 13$: $n = 77$, $\phi(n) = 60$, and $d = 37$ because $13 \times 37 = 481 = 8 \cdot 60 + 1$. Alice encrypts 5 and John encrypts 63 with the same public key $(13, 77)$; Bob decrypts both with $d = 37$. The table also has the class homework key ($n = 187$) and a small textbook key ($n = 33$); all values were recomputed.

p, q	n	$\phi(n)$	e	d	Encrypt	Decrypt
7, 11	77	60	13	37	$5^{13} \bmod 77 = 26$	$26^{37} \bmod 77 = 5$
7, 11	77	60	13	37	$63^{13} \bmod 77 = 28$	$28^{37} \bmod 77 = 63$
17, 11	187	160	7	23	$88^7 \bmod 187 = 11$	$11^{23} \bmod 187 = 88$
3, 11	33	20	7	3	$5^7 \bmod 33 = 14$	$14^3 \bmod 33 = 5$

For the slide key, the d-table (§**??**) on $(60, 13)$ has quotients $4, 1, 1, 1, 1, 2$ and ends with $r_1 = 1$, $t_1 = -23$, so $d = -23 + 60 = 37$. A negative t_1 is normal; add $\phi(n)$, never n.

1.6 Exam-format RSA numericals ▸

▸ **Likely this year:** Exactly this layout: $\phi(n)$, gcd check, d-table, two square-and-multiply tables, a check line.

Every exponentiation below uses repeated squaring (§**??**): write the exponent as a sum of powers of 2, square repeatedly (reducing mod n each time), then multiply the needed powers, reducing after each product.

WORKED EXAMPLE : Stallings/Forouzan, the class homework key

Q. In RSA, $p = 17$, $q = 11$, $e = 7$. Find n, $\phi(n)$ and the private key d. Encrypt $P = 88$ and show that decryption recovers it.

Keys. $n = 17 \times 11 = 187$; $\phi(n) = 16 \times 10 = 160$. Since $160 = 2^5 \cdot 5$ and 7 is prime, $\gcd(7, 160) = 1$, so $e = 7$ is valid. $d = 7^{-1} \bmod 160$:

q	r_1	r_2	r	t_1	t_2	t
22	160	7	6	0	1	−22
1	7	6	1	1	−22	23
6	6	1	0	−22	23	−160
	1	0		**23**	−160	

$r_1 = 1$ (inverse exists), $d = t_1 =$ **23**. Check: $7 \times 23 = 161 = 160 + 1$ ✓. Public key $(7, 187)$, private key 23.
Encrypt $C = 88^7 \bmod 187$, with $7 = 4 + 2 + 1$:

Power	Computation	mod 187
88^1		88
88^2	$88^2 = 7744 = 41 \cdot 187 + 77$	77
88^4	$77^2 = 5929 = 31 \cdot 187 + 132$	132

$88^7 = 88^4 \cdot 88^2 \cdot 88^1$: $132 \cdot 77 = 10164 \equiv 66$; $66 \cdot 88 = 5808 \equiv 11$. $\Rightarrow C =$ **11**.
Decrypt $P = 11^{23} \bmod 187$, with $23 = 16 + 4 + 2 + 1$:

Power	Computation	mod 187
11^1		11
11^2	$11^2 = 121$	121
11^4	$121^2 = 14641 = 78 \cdot 187 + 55$	55
11^8	$55^2 = 3025 = 16 \cdot 187 + 33$	33
11^{16}	$33^2 = 1089 = 5 \cdot 187 + 154$	154

$154 \cdot 55 = 8470 \equiv 55$; $55 \cdot 121 = 6655 \equiv 110$; $110 \cdot 11 = 1210 \equiv 88$. $\Rightarrow P =$ **88** ✓ (the original plaintext).

WORKED EXAMPLE : new, Forouzan exercise style (only n given)

Q. Bob's RSA public key is $(e, n) = (5, 221)$. (a) Find his private key. (b) Alice sends $P = 100$; find C. (c) Show Bob's decryption.
(a) Eve cannot factor a 1024-bit n, but 221 factors by trial division: $221 = 13 \times 17$. So $\phi(n) = 12 \times 16 = 192 = 2^6 \cdot 3$, and $\gcd(5, 192) = 1$. $d = 5^{-1} \bmod 192$:

q	r_1	r_2	r	t_1	t_2	t
38	192	5	2	0	1	−38
2	5	2	1	1	−38	77
2	2	1	0	−38	77	−192
	1	0		**77**	−192	

$d =$ **77**. Check: $5 \times 77 = 385 = 2 \cdot 192 + 1$ ✓.
(b) $5 = 4 + 1$: $100^2 = 10000 = 45 \cdot 221 + 55$; $55^2 = 3025 = 13 \cdot 221 + 152$, so $100^4 \equiv 152$. $C = 152 \cdot 100 = 15200 = 68 \cdot 221 + 172 \Rightarrow C =$ **172**.
(c) $P = 172^{77} \bmod 221$, with $77 = 64 + 8 + 4 + 1$:

Power	Computation	mod 221
172^1		172
172^2	$172^2 = 29584 = 133 \cdot 221 + 191$	191
172^4	$191^2 = 36481 = 165 \cdot 221 + 16$	16
172^8	$16^2 = 256 = 221 + 35$	35
172^{16}	$35^2 = 1225 = 5 \cdot 221 + 120$	120
172^{32}	$120^2 = 14400 = 65 \cdot 221 + 35$	35
172^{64}	$35^2 \equiv 120$ (as above)	120

$172^{64} \cdot 172^8 = 120 \cdot 35 = 4200 = 19 \cdot 221 + 1 \equiv 1$; $1 \cdot 16 = 16$; $16 \cdot 172 = 2752 = 12 \cdot 221 + 100$. $\Rightarrow P =$ **100** ✓.

1.7 Choosing e; why $e = 2$ never works ▸

▸ **Likely this year:** "Why is $e = 2$ not allowed? or "is $e = 3$ valid for $p = 19$, $q = 23$? (1–2 marks).

Encryption cost depends on the number of bits and of 1-bits in e, so a small e with few 1-bits makes encryption cheap. The standard choice is $e = 2^{16} + 1 = 65537$: it is prime (so it is coprime to $\phi(n)$ unless a prime factor of $\phi(n)$ is 65537 itself), its binary form 1 0000 0000 0000 0001 needs only 16 squarings and one multiplication, and it is large enough to defeat the low-exponent attacks below. The private exponent d is then about as long as n, so decryption is the slow side (which the CRT speeds up).

$e = 2$ **(or any even e) is invalid.** Since p and q are odd primes, $p - 1$ and $q - 1$ are even, so $\phi(n)$ is even (in fact divisible by 4). Then $\gcd(2, \phi(n)) = 2 \neq 1$, e is not in $\mathbb{Z}^*_{\phi(n)}$, and no d exists. The cause is that squaring is not one-to-one: x and $n - x$ have the same square, so $C = P^2$ cannot be uniquely undone. The same check catches a Forouzan exercise: $p = 19$, $q = 23$, $e = 3$ gives $\phi(n) = 18 \cdot 22 = 396 = 3 \cdot 132$, so $\gcd(3, 396) = 3$ and $e = 3$ is not allowed for this n.

Trap: the usual RSA slips

d is the inverse of e modulo $\phi(n)$, never modulo n (for $(7, 187)$ the answer is 23; $7^{-1} \bmod 187 = 107$ is useless). Check $\gcd(e, \phi(n)) = 1$ before running the table. The plaintext must satisfy $P < n$. Encrypt with the *receiver's* public key. Reduce mod n after every multiplication, and turn a negative t_1 into $t_1 + \phi(n)$.

1.8 Faster decryption with the CRT ▸

▸ **Likely this year:** "Decrypt using the Chinese remainder theorem" on a small key, or "why is CRT decryption about 4 times faster".

Decryption is slow because d and n are both full size. Bob, however, knows p and q, so he can work modulo each prime separately and combine the answers with the CRT (§**??**). Modulo p the exponent can be reduced: since $C^{p-1} \equiv 1 \pmod p$ (Fermat), $C^d \equiv C^{d \bmod (p-1)} \pmod p$. Bob computes

$$d_p = d \bmod (p-1), \quad d_q = d \bmod (q-1), \qquad P_p = C^{d_p} \bmod p, \quad P_q = C^{d_q} \bmod q,$$

and then the unique $P < n$ with $P \equiv P_p \pmod p$ and $P \equiv P_q \pmod q$:

$$P = \left(P_p \cdot q \cdot (q^{-1} \bmod p) + P_q \cdot p \cdot (p^{-1} \bmod q)\right) \bmod n.$$

Each half uses a modulus of half the length (a multiplication costs about $1/4$) and an exponent of half the length (half as many steps), so each half costs about $1/8$ of the full exponentiation and both together about $1/4$: decryption is roughly **4 times faster**. Only the key owner can use it, since it needs p and q.

WORKED EXAMPLE : CRT decryption of the previous ciphertext

Q. With $p = 13$, $q = 17$, $d = 77$, decrypt $C = 172$ using the CRT.
$d_p = 77 \bmod 12 = 5$, $d_q = 77 \bmod 16 = 13$; $C_p = 172 \bmod 13 = 3$, $C_q = 172 \bmod 17 = 2$.
$P_p = 3^5 \bmod 13 = 243 \bmod 13 = 9$. $P_q = 2^{13} \bmod 17$: $2^4 = 16 \equiv -1$, $2^8 \equiv 1$, so $2^{13} = 2^8 \cdot 2^4 \cdot 2 \equiv -2 \equiv 15$.
$q^{-1} \bmod p = 17^{-1} \bmod 13 = 4^{-1} \bmod 13 = 10$ $(4 \cdot 10 = 40 = 3 \cdot 13 + 1)$; $p^{-1} \bmod q = 13^{-1} \bmod 17 = 4$ $(13 \cdot 4 = 52 = 3 \cdot 17 + 1)$.
$P = 9 \cdot 17 \cdot 10 + 15 \cdot 13 \cdot 4 = 1530 + 780 = 2310 = 10 \cdot 221 + 100 \Rightarrow P =$ **100** ✓, the same as $172^{77} \bmod 221$, with exponents 5 and 13 instead of 77.

1.9 Attacks on RSA ▸

▸ **Likely this year:** "Explain any two attacks on RSA" or "why must p and q be large / not close / why OAEP".

No devastating attack on correctly used RSA is known; every known attack exploits a weak choice of parameters or a careless use of textbook RSA. The slide's list, each with the reason it works:

- **Factorization.** If Eve factors n she computes $\phi(n)$ and then $d = e^{-1} \bmod \phi(n)$, exactly as Bob did. The whole security rests on n being too large to factor.
- **Chosen-ciphertext.** RSA is multiplicative: $(P_1 P_2)^e = P_1^e P_2^e$. To decrypt C, Eve picks X, asks Bob to decrypt the innocent-looking $Y = C \cdot X^e \bmod n$, receives $Z = Y^d = P \cdot X$, and computes $P = Z \cdot X^{-1}$. With $(7, 187)$, $C = 11$, $X = 2$: $Y = 11 \cdot 128 \bmod 187 = 99$, $Z = 99^{23} \bmod 187 = 176$, $P = 176 \cdot 94 \bmod 187 = 88$ ✓.
- **Low encryption exponent.** If e is small and $P^e < n$, no reduction happens and $P = \sqrt[e]{C}$ over the integers ($e = 3$, $n = 187$, $P = 5$: $C = 125$, $\sqrt[3]{125} = 5$). **Coppersmith's** theorem extends this to finding small unknown parts of P. **Broadcast**: if the same P goes to e receivers with moduli $n_1, \ldots, n_e$, the CRT gives $P^e \bmod n_1 \cdots n_e$, which equals P^e because $P^e < n_1 \cdots n_e$, so an integer root reveals P (with $e = 3$, $n = 55, 391, 1189$ and $P = 42$: ciphertexts $3, 189, 370$, CRT gives $74088 = 42^3$). **Related messages** (Franklin–Reiter) and **short pad** attacks similarly recover messages that are linearly related or padded with too few random bits.
- **Low decryption exponent.** A small d speeds decryption but Wiener's attack recovers d from e/n by continued fractions when $d < \frac{1}{3} n^{1/4}$. A **revealed** d is worse than it looks: n with one pair (e, d) lets anyone factor n, so all keys on that modulus are dead.
- **Plaintext attacks.** *Short message*: if P comes from a small set (a PIN), Eve encrypts every candidate with the public key and compares, since textbook RSA is deterministic. *Cyclic*: C is a permutation of $\mathbb{Z}_n$, so re-encrypting $C, C^e, C^{e^2}, \ldots$ eventually returns to C, and the value just before C is P; with $(7, 33)$ and $C = 14$: $14 \to 20 \to 26 \to 5 \to 14$, so $P = 5$. *Unconcealed*: some P satisfy $P^e \equiv P$ (always 0, 1, $n - 1$); their number is

$(1 + \gcd(e-1, p-1))(1 + \gcd(e-1, q-1))$, which for the slide key $(13, 77)$ is $7 \cdot 3 = 21$ messages (e.g. $10^{13} \equiv 10$), because $e - 1 = 12$ is a multiple of $p - 1 = 6$.

- **Common modulus.** If two users share n with exponents e_1, e_2, $\gcd(e_1, e_2) = 1$, and the same P is sent to both, Eve finds $se_1 + te_2 = 1$ and gets $C_1^s C_2^t = P^{se_1+te_2} = P$. With $n = 187$, $e_1 = 3$, $e_2 = 7$, $P = 5$: $C_1 = 125$, $C_2 = 146$, $s = -2$, $t = 1$, $P = (125^{-1})^2 \cdot 146 = 3^2 \cdot 146 = 1314 \equiv 5$ ✓. Also, any user of the shared n can factor it from his own (e, d).
- **Implementation (timing, power).** Square-and-multiply does an extra multiplication for every 1-bit of d, so the decryption time or power trace leaks the bits of d. Defences: constant-time exponentiation, random delays, or *blinding* (decrypt $C \cdot r^e$, then multiply by r^{-1}).

1.10 Recommendations

Each recommendation blocks one of the attacks above.

- n at least **1024 bits** (about 309 decimal digits; 2048 today), so each of p, q at least **512 bits** (about 154 digits). This defeats factorization.
- p and q must not be close to each other, otherwise Fermat factoring ($n = a^2 - b^2$ with $a \approx \sqrt{n}$) finds them quickly. Both $p - 1$ and $q - 1$ should have a large prime factor (against Pollard's $p - 1$ method), and p/q should not be close to a fraction with small numerator and denominator.
- The modulus must not be shared between users (common-modulus attack).
- $e = 2^{16} + 1$ or an integer close to it (against low-exponent attacks, while keeping encryption fast).
- If d leaks, Bob must change n, e and d, since one (e, d) pair reveals the factors of n.
- Messages must be padded with **OAEP** (Optimal Asymmetric Encryption Padding). OAEP mixes the message with a fresh random value r through two hash-based masks ($M_1 = P \oplus G(r)$, $M_2 = r \oplus H(M_1)$) and encrypts $M_1 \| M_2$. This makes RSA probabilistic (same P, different C each time), fills the block so $P^e > n$, and destroys the multiplicative structure, blocking the short-message, broadcast, cyclic-style and chosen-ciphertext tricks.

Corrections to the notes / slides

- Slide on trapdoor functions: "there is not a polynomial time solution" to factoring should read *no polynomial-time classical algorithm is known*; it is not proven impossible (Shor's quantum algorithm is polynomial).
- Slide on RSA size: a 512-bit prime has 154 or 155 decimal digits (2^{511} has 154, 2^{512} has 155); 1024 bits is 309 digits, as stated.

Check your understanding

1. **Differentiate symmetric and asymmetric key cryptography.** Symmetric uses one shared secret key for both directions, needs $N(N-1)/2$ keys and a secure key exchange, and is fast. Asymmetric uses a public/private pair per user, needs only N pairs and no prior secret, rests on number-theoretic problems, and is slow, so it is used for key exchange and signatures.
2. **Why is $e = 2$ invalid?** $\phi(n) = (p-1)(q-1)$ is even, so $\gcd(2, \phi(n)) \neq 1$ and 2 has no inverse modulo $\phi(n)$; squaring maps x and $n - x$ to the same value, so it cannot be inverted uniquely.
3. **Why is d computed modulo $\phi(n)$ and not n?** Exponents of an element of $\mathbb{Z}_n^*$ repeat with period dividing $\phi(n)$ (Euler), so $P^{ed} = P$ needs $ed \equiv 1 \pmod{\phi(n)}$; a congruence modulo n says nothing about exponents.
4. **Why must $P < n$?** Results are only defined modulo n, so P and $P + n$ encrypt to the same C and decryption returns P mod n.
5. **Given $n = 187$ and $e = 17$, find d.** $187 = 11 \cdot 17$, $\phi = 160$, $17^{-1} \bmod 160 = 113$ ($17 \cdot 113 = 1921 = 12 \cdot 160 + 1$).
6. **Given $n = 3937$ and $e = 17$, find d.** $3937 = 31 \cdot 127$, $\phi = 30 \cdot 126 = 3780$, $d = 3113$ ($17 \cdot 3113 = 52921 = 14 \cdot 3780 + 1$).
7. **Why does knowing $\phi(n)$ break RSA?** It gives d directly, and also $p+q = n - \phi(n) + 1$, so p, q are roots of $x^2 - (p+q)x + n$.
8. **Why is RSA not used to encrypt bulk data?** It is about a thousand times slower than AES and textbook RSA is deterministic; it encrypts a session key instead (hybrid encryption).
9. **What does OAEP add?** Randomness and structure: the same message encrypts differently each time and malleability is destroyed, which stops short-message, broadcast and chosen-ciphertext attacks.
10. **Why is CRT decryption only for Bob?** It needs p and q, which only the key owner knows; the gain is about a factor of 4.